

Functions can have docstrings, too. Notice the docstring for `main()` below the `def` statement. You can view this docstring by running `import towerofhanoi` and `help(towerofhanoi.main)` from the interactive shell.

Next, we write a comment that extensively describes the data structure we use to represent the tower, because it forms the core of how this program works:

```
"""The towers dictionary has keys "A", "B", and "C" and values
that are lists representing a tower of disks. The list contains
integers representing disks of different sizes, and the start of
the list is the bottom of the tower. For a game with 5 disks,
the list [5, 4, 3, 2, 1] represents a completed tower. The blank
list [] represents a tower of no disks. The list [1, 3] has a
larger disk on top of a smaller disk and is an invalid
configuration. The list [3, 1] is allowed since smaller disks
can go on top of larger ones."""
towers = {"A": copy.copy(SOLVED_TOWER), "B": [], "C": []}
```

We use the `SOLVED_TOWER` list as a *stack*, one of the simplest data structures in software development. A stack is an ordered list of values altered only through adding (also called *pushing*) or removing (also called *popping*) values from the *top* of the stack. This data structure perfectly represents the tower in our program. We can turn a Python list into a stack if we use the `append()` method for pushing and the `pop()` method for popping, and avoid altering the list in any other way. We'll treat the end of the list as the top of the stack.

Each integer in the `towers` list represents a single disk of a certain size. For example, in a game with five disks, the list `[5, 4, 3, 2, 1]` would represent a full stack of disks from the largest (5) at the bottom to the smallest (1) at the top.

Notice that our comment also provides examples of a valid and invalid tower stack.

Inside the `main()` function, we write an infinite loop that runs a single turn of our puzzle game:

```
while True: # Run a single turn on each iteration of this loop.
    # Display the towers and disks:
    displayTowers(towers)
```
